

Week 1

1. Базовые понятия RL

Обучение с подкреплением (RL) направлено на поиск оптимальной стратегии методом проб и ошибок. Агенту требуется обратная связь от среды на его действия, при этом эти действия напрямую влияют на будущие наблюдения.

Процесс описывается как Марковский процесс принятия решений (MDP):

- Взаимодействующие сущности: Агент (Agent) и Среда (Environment).
- Состояния среды: $s \in S$.
- Действия агента: $a \in A$.
- Награды: $r \in \mathbb{R}$.
- Динамика среды: $P(s_{t+1}|s_t, a_t)$. Выполняется Марковское свойство: $P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}) = P(s_{t+1}|s_t, a_t)$.

Суммарная награда за сессию: $R = \sum_t r_t$. Политика агента: $\pi(a|s) = P(\text{take action } a \text{ in state } s)$. Задача — найти политику с наибольшей ожидаемой наградой: $\pi(a|s) : E_\pi[R] \rightarrow \max$.

2. Метод кросс-энтропии

Общая логика: сыграть несколько сессий, обновить политику, повторить.

Табличный метод

Применяется для конечных пространств. Политика задается матрицей $\pi(a|s) = A_{s,a}$.

1. Сэмплируются N игр с текущей политикой.
2. Отбираются M лучших сессий (элитных): $Elite = [(s_0, a_0), (s_1, a_1), \dots, (s_k, a_k)]$.
3. Политика агрегируется по состояниям (вычисляется доля успешных действий):

$$\pi(a|s) = \frac{\sum_{s_i, a_i \in Elite} [s_i = s][a_i = a]}{\sum_{s_i, a_i \in Elite} [s_i = s]}$$

Аппроксимированный метод (с нейронной сетью)

Применяется, когда пространство состояний слишком велико. Нейронная сеть предсказывает распределение $\pi_w(a|s)$.

1. Собираются все пары состояние-действие из M лучших сессий: $Elite = [(s_0, a_0), \dots, (s_k, a_k)]$.
2. Максимизируется правдоподобие действий в элитных играх:

$$\pi = \arg \max_{s_i, a_i \in Elite} \log \pi(a_i|s_i)$$

3. Веса сети W обновляются градиентным шагом:

$$W_{i+1} = W_i + \alpha \nabla \left[\sum_{s_i, a_i \in Elite} \log \pi_{W_i}(a_i|s_i) \right]$$

Week 2

1. Функции $v(s)$ и $q(s, a)$

State-value function, $v(s)$: ожидаемая возвращаемая ценность (суммарная дисконтированная награда) при условии нахождения в состоянии s .

- Смысл: показывает ценность следования выбранной политике π , начиная из состояния s .
- Формула: $v_\pi(s) \triangleq \mathbb{E}_\pi[G_t | S_t = s]$.

Action-value function, $q(s, a)$: ожидаемая возвращаемая ценность при условии нахождения в состоянии s и совершения конкретного действия a .

- Смысл: показывает ценность совершения действия a в состоянии s , если после этого агент продолжит следовать политике π .
- Формула: $q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$.

2. Метод Policy Iteration

Алгоритм поиска оптимальной политики, который циклично чередует два процесса: оценку текущей политики и её улучшение.

1. **Инициализация:** задаются произвольные значения функции ценности $V(s)$ и политики $\pi(s)$ для всех состояний среды $s \in \mathcal{S}$.
2. **Policy Evaluation (Оценка политики):** для каждого состояния s итеративно вычисляется функция ценности с использованием уравнения ожидания Беллмана. Обновление повторяется до сходимости (пока изменение не станет меньше порога θ).

$$V(s) \leftarrow \sum_{s', r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$$

3. **Policy Improvement (Улучшение политики):** для каждого состояния s политика обновляется выбором «жадного» действия, максимизирующего ожидаемую награду:

$$\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Условие остановки: если действия не изменились, алгоритм завершается (оптимальная политика найдена). Если изменились — возврат к шагу 2.

Week 3

1. Q-learning и SARSA

Оба алгоритма применяются для обучения без использования модели среды (Model-free RL). Они обновляют функцию ценности $Q(s, a)$ итеративно по базовому правилу:

$$Q(s, a) \leftarrow \alpha \cdot \hat{Q}(s, a) + (1 - \alpha)Q(s, a)$$

Q-learning (Off-policy)

- Формула оценки: $\hat{Q}(s, a) = r(s, a) + \gamma \max_{a'} Q(s', a')$.
- Алгоритм оценивает следующее состояние, предполагая выбор наилучшего (жадного) действия, независимо от того, какое действие агент совершит на самом деле.
- Так как это off-policy метод, он может обучаться оптимальной политике на исторических данных или случайных действиях (в том числе через буфер Experience replay).

SARSA (On-policy)

- Формула оценки: $\hat{Q}(s, a) = r(s, a) + \gamma Q(s', a')$.
- Здесь a' — это реальное следующее действие, которое агент выбрал согласно своей текущей политике.
- Это on-policy метод, он оценивает именно ту политику, которой агент следует сейчас. В средах с риском (например, Cliff world) SARSA находит более безопасный путь, учитывая вероятность своих случайных ошибок.

2. Exploration vs Exploitation

Это концепция поиска баланса между применением уже накопленных знаний для получения награды (Exploitation) и попытками найти новые, потенциально более выгодные решения (Exploration). Если всегда выбирать только известные «лучшие» действия, агент может застрять в локальном оптимуме.

Основные стратегии исследования:

1. **ϵ -greedy (эпсилон-жадная):** с заданной вероятностью ϵ агент выбирает случайное действие, а в остальных случаях $(1 - \epsilon)$ — оптимальное.
2. **Softmax:** вероятность выбора действия пропорциональна функции softmax от его Q-значений:

$$\pi(a|s) = \text{softmax}\left(\frac{Q(s, a)}{\tau}\right)$$

где τ (температура) контролирует уровень случайности.

Чтобы алгоритм сошелся к оптимальной политике, уровень исследования необходимо постепенно снижать (например, плавно уменьшать ϵ до нуля).

Week 4

1. DQN (Deep Q-Network)

DQN применяет нейронную сеть (модель с параметрами Θ) для аппроксимации функции ценности $Q(s, a)$. Это позволяет алгоритму работать с огромными пространствами состояний (например, пикселями экрана видеоигры).

- **Вход:** состояние среды s (например, набор последних кадров).
- **Выход:** предсказанные значения $Q(s, a)$ для каждого возможного действия a .
- **Функция потерь:** минимизируется среднеквадратичная ошибка (MSE) между предсказанием сети и таргетом:

$$L(\Theta) = \mathbb{E}_{s,a} \left[\left(Q(s, a, \Theta) - (r + \gamma \max_{a'} Q(s', a', \Theta^-)) \right)^2 \right]$$

2. Experience Replay (Буфер опыта)

Метод решает две проблемы глубокого обучения в RL: сильную корреляцию последовательных кадров (не i.i.d. данные) и потерю редкого или старого опыта.

- **Принцип:** Агент сохраняет историю своих взаимодействий со средой в виде кортежей $\langle s, a, r, s' \rangle$ в большой кольцевой буфер (Replay buffer).
- **Обучение:** Вместо обучения на последнем шаге, градиентный спуск делается на случайных мини-батчах из этого буфера.
- **Ограничение:** Метод требует использования off-policy алгоритмов (как Q-learning), потому что опыт в буфере собран старыми (и более слабыми) версиями политики агента.

3. Target Network (Целевая сеть)

В классическом Q-learning таргет (эталон, к которому стремится сеть) зависит от весов самой сети. При обновлении весов смещается и таргет, что приводит к нестабильности (проблема «движущейся цели»).

- **Решение:** Использование отдельной копии сети с «замороженными» весами Θ^- для вычисления $Q(s', a')$.
- **Hard target network:** Веса целевой сети полностью копируются из основной сети каждые N шагов.
- **Soft target network:** Веса целевой сети обновляются понемногу на каждом шаге с помощью экспоненциального сглаживания:

$$\Theta^- \leftarrow (1 - \alpha)\Theta^- + \alpha\Theta$$

4. Double DQN

Решает проблему переоценки (overestimation) Q-значений. В стандартном DQN использование оператора $\max$ для оценки будущей награды приводит к накоплению статистического шума и завышению ценности состояний.

- **Логика:** Процесс разбивается на две части: выбор лучшего действия ($\arg\max$) и оценка ценности этого действия. Их выполняют разные сети.
- Основная сеть (Θ) выбирает действие, а целевая сеть (Θ^-) оценивает его полезность.
- **Формула таргета Double DQN:**

$$y_{ddqn} = r + \gamma Q(s', \arg \max_{a'} Q(s', a', \Theta), \Theta^-)$$

Week 6

1. Policy gradient

Градиент стратегии (Policy gradient) — это подход, при котором агент напрямую обучается оптимальной стратегии $\pi_\theta(a|s)$. Метод позволяет избежать вычисления ценности всех состояний и направлен на прямую максимизацию ожидаемой награды.

- Для аналитического вычисления используется логарифмический трюк: $\nabla \log \pi(z) = \frac{1}{\pi(z)} \nabla \pi(z)$.
- Итоговая формула градиента ожидаемой награды: $\nabla J = \mathbb{E}[\nabla \log \pi_\theta(a|s) Q(s, a)]$.

2. REINFORCE

Базовый алгоритм семейства Policy Gradient, который обновляет веса на основе полных игровых сессий.

1. Сэмплируются N сессий с использованием текущей политики $\pi_\theta(a|s)$.
2. Вычисляется градиент: $\nabla J \approx \frac{1}{N} \sum_i \sum_{s,a} \nabla \log \pi_\theta(a|s) \cdot Q(s, a)$, где $Q(s, a)$ заменяется на фактическую суммарную награду G_t с данного шага.
3. Веса сети обновляются шагом градиентного подъема: $\theta_{i+1} \leftarrow \theta_i + \alpha \nabla J$.

Для уменьшения дисперсии (variance) градиента из оценки вычитается базовая линия $b(s)$ (baseline), например, $V(s)$. Это не меняет направление градиента, но делает процесс обучения более стабильным.

3. Actor-critic

Архитектура, которая одновременно обучает две нейронные сети: стратегию $\pi_\theta(a|s)$ (Actor) и функцию ценности $V_\theta(s)$ (Critic).

- Модель Critic используется для ускорения обучения Actor за счет вычисления функции преимущества (Advantage): $A(s, a) = r + \gamma V(s') - V(s)$.
- Обновление Actor (улучшение стратегии): $\nabla J_{actor} \approx \frac{1}{N} \sum \nabla \log \pi_\theta(a|s) \cdot A(s, a)$.
- Обновление Critic (улучшение оценки функции ценности): $L_{critic} \approx \frac{1}{N} \sum (V_\theta(s) - [r + \gamma V(s')])^2$.

Week 7

1. Применение RNN для генерации последовательностей

Архитектуры Encoder-Decoder на базе RNN применяются для задач преобразования последовательностей: машинного перевода, генерации описаний к изображениям (Image captioning) и создания диалоговых систем.

- **Механика:** Кодировщик (Encoder) сжимает входную последовательность в вектор скрытого состояния h . Декодировщик (Decoder) генерирует выходную последовательность шаг за шагом, опираясь на h и свои предыдущие действия (a_{t-1}).
- **Проблема сдвига распределения (Distribution shift):** При классическом обучении с учителем (Supervised Learning) декодировщик предсказывает следующий токен, имея в качестве контекста идеальную эталонную последовательность. Во время реальной работы (Inference) эталона нет, и модель опирается на собственные сгенерированные токены. Из-за этого любая ошибка приводит к переходу в нестандартное состояние, и качество последующей генерации резко падает.

2. Применение RL к дообучению моделей генерации

В отличие от обучения с учителем, RL позволяет обучать модель на её собственных предсказаниях. Это решает проблему сдвига распределения и позволяет оптимизировать недифференцируемые метрики (например, человеческие предпочтения или метрику CIDEr).

Основные алгоритмы:

- **SCST (Self-Critical Sequence Training):** Улучшение алгоритма REINFORCE для генерации текста. Для снижения дисперсии градиента используется вычитание базовой линии. В качестве базовой линии берется награда за последовательность, сгенерированную жадным алгоритмом (inference mode).

$$A(s, a) = R(s, a) - R(s, a_{inference}(s))$$

Модель обучается только на тех сэмплированных ответах, которые оказались метрически лучше жадной генерации.

- **RLHF (Reinforcement Learning from Human Feedback):** Метод выравнивания поведения языковых моделей (используется в ChatGPT), состоящий из трех этапов:
 1. *SFT (Supervised Fine-Tuning):* Дообучение модели на качественных демонстрациях от разметчиков.
 2. *Reward Model (RM):* Обучение отдельной модели-оценщика. Разметчики ранжируют несколько ответов модели, и RM учится предсказывать этот человеческий выбор (награду).
 3. *RL-оптимизация:* Языковая модель дообучается с помощью алгоритма PPO, стремясь максимизировать оценку, которую выдает RM.
- **DPO (Direct Preference Optimization):** Более математически простая альтернатива RLHF. Исключает этапы создания Reward Model и применения алгоритма PPO. Модель оптимизируется напрямую на датасете с предпочтениями (какой ответ лучше, какой хуже) с помощью модифицированной функции потерь.